

Front End

Structure

The main structure of my front end follows the MVC design pattern. This design will split my front end into 3 separate components. Controllers, Views, and the Model. Users will make requests to the Controllers. The Controllers will then call the Model to access this information from the database. Upon receiving this information from the Model, the Controller will send this to the View files. As implied through the name the view files are what the user views ie. the GUI. Additionally this works very well with the DAO design pattern. Within the middle layer of my 3 tier application there will be classes such as the user class, and the listing class. Both of these will be used to help the controllers send information to the views and for the views to display information. With the DOA design pattern I'll have these classes, but each class will have their own DOA class which will act as database getters for the class. The user class will never call the database itself, rather it will class the methods within the userDAO class to make these calls. Typically DOA implementations have the actual object (not the DAO object), instantiated off of an interface. But I feel like this is unnecessary in my case. I could add a user interface, and a listing interface, but again I don't think it's necessary unless I were to add different types of listings or users. Which could be a possibility if I wanted to add an auction listing type, but for now I want to stick with a set price until I can ensure it works perfectly.

Functionality

Currently I have 0 functionality within my front end. I have mainly worked on the database portion of my project currently. This will be my next goal for report 2, getting my project into a working state where my project should be closing in upon completion. While there is no real functionality, I do have a large chunk of the "skeleton" of what my project will look like done. I also have done some slight testing ensuring my database connection works and I am able to use CRUD functionality from my middle layer, (these functions have been removed as it was before my stored procedures were added).

Back End

Tables

- *User*
 - id
 - name
 - password (will be hashed in java section)
- *Listing*
 - id
 - Name
 - Description
 - Picture (not set up yet)
 - Activelisting bool
 - Start time
 - End time
 - Price
 - State
 - Town
 - Long & Lat (added to be interactive with google maps api later on)
 - quantity
 - User_id (fk to show who owns said listing)
- *Listing Category*
 - Id (composite key built on listing id and category id)
- *Listing Archive*
 - Cloned listing table but these have expired
- *Category*
 - Id
 - name
- *Order History*
 - All of the purchase information

Stored Procedures

- *Create Listing*
 - Does as name says no checks
- *Buy Listing*
 - Does as name says, no checks, will create listing and add it to listing table, and listing category table
- *Delete listing*
 - Does as name says, no checks (will be handled in GUI)

Triggers

- *Prevent negative prices*
 - Does as name says

Events

- *Expire Active Listings*
 - Once a day DB will check the 'end_time' field and make sure it hasn't past, if it has it will then turn the 'active' bool to false
- *Delete Old Listings*
 - Once a day DB will check and see if 'active' bool has been false for one month, the only way this bool will ever be false is if the previous event has occurred